# HUSKEY POCKET

## Security and Architecture Whitepaper

*ESP32-S3 hardware password manager*

**Revision 4 — current architecture, tests, and limitations**

Technical document on design, limits, and threat model

# Contents

**Contents** ..... 2
**About this revision** ..... 3
**1. Introduction** ..... 3
**2. Threat Model** ..... 3
**3. Overall Architecture** ..... 4
**4. Storage Architecture** ..... 4
**5. Cryptographic Model** ..... 5
**6. Key Management** ..... 6
**7. Authentication Flow** ..... 6
**8. Brute-force Protection** ..... 6
**9. Factory Reset Protection (FRP)** ..... 7
**10. Decoy PIN** ..... 7
**11. Lost Mode (Dead Man's Switch)** ..... 7
**12. Snatch Lock** ..... 8
**13. Export and Import** ..... 8
**14. Firmware Updates** ..... 9
**15. Physical Protection** ..... 9
**16. Bluetooth** ..... 9
**17. USB Storage Mode** ..... 10
**18. TOTP and Diagnostic Policy** ..... 10
**19. Scope and Design Decisions** ..... 11
**20. Stability and Memory Verification** ..... 11
**21. Limitations** ..... 11
**22. Revision 4 Changes** ..... 12
**23. Revision 3 Changes** ..... 12
**24. Earlier Code Changes** ..... 12
**25. Frequently Asked Questions** ..... 13
**26. Glossary** ..... 14

## About this revision

This revision describes the current firmware architecture and the functional tests performed. Two AI-assisted memory reviews examined leaks and memory-management issues; all 16 reported tests passed on the code evaluated at that time. No security audit has been performed. These reviews do not certify the absence of vulnerabilities.

> *Secure Boot V2 is enabled on the author's unit. Flash-encryption settings and recovery options depend on each unit's provisioning. This document is not a security audit or firmware certification.*

# 1. Introduction

### What is Huskey Pocket?

Huskey Pocket is a hardware password manager: a standalone physical device with no internet or cloud connection that stores encrypted credentials and types them on a computer by emulating a Bluetooth or USB keyboard. The user does not type or copy passwords; after verifying a PIN on its touchscreen, the device types them on the user's behalf.

### What problem does it solve?

Software password managers (browser extensions or cloud apps) depend on the security of the operating system and browser where they run. If that computer is compromised, the manager may be exposed. Huskey Pocket separates credential storage from the computer that uses the credentials. The encrypted vault is stored locally. USB/Bluetooth functions send the fields the user asks the device to type.

### Design philosophy

- No cloud, no account, and no automatic synchronization. The device owner is the sole point of trust.
- Each PIN attempt pays the full PBKDF2 key-derivation cost. There is no cheap shortcut for checking candidate PINs.
- Fail safely. When uncertain, such as when the RTC is out of sync, the system prefers not to act rather than risk erasing data by mistake.
- The firmware has no Wi-Fi or cloud-sync features. It uses USB and Bluetooth as keyboard channels and provides USB access to the microSD card in an exclusive mode.
- Sensitive material does not leave the device through diagnostic channels. No secret, key, or code is logged over the serial port.

# 2. Threat Model

This is the most important section: it defines what Huskey Pocket is designed to protect against and, just as clearly, what it does not promise.

### Protects against

- Device theft. The vault remains encrypted without the PIN; there is no factory recovery key that bypasses it.
- PIN brute-force attacks. Each attempt pays the full PBKDF2 cost (50,000 iterations), and the system starts a factory reset after five consecutive failures. The counter is saved before PIN verification.

- Prolonged loss of the device. Lost Mode erases the contents after N days without a genuine unlock if the user enabled it.
- Theft followed by reconfiguration. Factory Reset Protection (FRP) requires the original PIN after any factory reset, using a persistent attempt counter that is not reset by power loss.
- Decoy PIN. A decoy PIN can open a separate, empty vault. The code derives both real and decoy candidate keys before verification; resistance to timing attacks has not been audited.
- Physical snatching while unlocked. The firmware polls the inertial sensor and locks the device if readings exceed configured thresholds. Detection is software-based, and no fixed response time is guaranteed.
- Unauthorized access to storage. Each sensitive field is encrypted with AES-256-CBC and its own IV; the salt is used to derive the key, and decryption validates the complete padding before accepting data.

### Does not protect against

- A user who voluntarily reveals their PIN after being deceived (for coercion, the decoy PIN is available).
- Malware already installed on the computer used as a keyboard. Huskey Pocket types credentials as the user would and cannot determine whether the receiving system is compromised.
- Sophisticated chip-laboratory attacks (advanced side-channel analysis, decapsulation). Secure Boot is enabled on the author's unit. Flash Encryption and the remaining eFuse state have not been verified.
- Social engineering directed at the user outside the device.

## 3. Overall Architecture

Huskey Pocket runs on a Waveshare ESP32-S3-Touch-LCD-2.8. Its main components are:

| Component | Details |
|---|---|
| MCU | ESP32-S3, with hardware cryptographic acceleration through mbedTLS. |
| Storage | Internal Flash with NVS for settings and control values, plus a LittleFS partition for encrypted vault records. |
| RTC | Dedicated PCF85063ATL; requires an external backup battery to retain time while off. Used for TOTP and Lost Mode. |
| IMU | QMI8658C (accelerometer + gyroscope) on the I2C bus shared with the RTC. Used by Snatch Lock. |
| microSD | Used for encrypted import/export and manually activated mass-storage mode only. |
| Display | ST7789 240 × 320 with CST328 touch controller on an independent I2C bus. |
| Bluetooth | NimBLE, keyboard HID profile only. Numeric confirmation pairing; typing only over an encrypted and authenticated link. |
| USB | Native HID keyboard or exclusive mass storage; exits automatically when the cable is disconnected. |

## 4. Storage Architecture

### NVS

- Vault namespace: control value for the real PIN (derived, not the PIN itself) and decoy PIN.

- Security namespace: failed-attempt counter, persistently written before each verification.
- FRP namespace: FRP PIN control value, permanent-lock flag, and FRP attempt counter. It is never erased by a factory reset.
- Decoy-vault namespaces: used for configuration, but save operations in decoy mode are fully simulated and never write real data. They are cleared during factory reset.
- Preferences namespace: theme, language, auto-lock, layout, Lost Mode duration, last real-unlock timestamp, Snatch Lock level, and USB read-only preference. It contains no secrets.

### LittleFS

Contains vault records in a fixed-size binary format. Each sensitive field (name, username, password, secret, notes) is encrypted individually with its own IV and PKCS#7 padding. It also contains three persistent salt files (real PIN, decoy PIN, and FRP), each generated once.

### What is and is not encrypted?

| Encrypted with AES-256-CBC | Plaintext (low-risk metadata) |
|---|---|
| Passwords, secrets, notes, TOTP secrets, and each entry's name. | Category IDs, interface preferences, attempt counter, Lost Mode and Snatch Lock settings, and their timestamps. |

Plaintext metadata was considered low risk because it cannot reconstruct credentials or let someone outside the device distinguish the real vault from the decoy.

## 5. Cryptographic Model

| Parameter | Value |
|---|---|
| Symmetric encryption | AES-256-CBC (mbedTLS) |
| Padding | PKCS#7, with full padding-byte validation during decryption |
| Key derivation | PBKDF2-HMAC-SHA256 |
| Iterations (device PIN) | 50,000 |
| Iterations (encrypted export) | 100,000, with an export password separate from the PIN. The importer rejects files declaring fewer iterations. |
| Salt | 16 random bytes, generated once and persisted in LittleFS. |
| IV | 16 random bytes for each encryption operation, stored beside the ciphertext and never reused. |

CBC was chosen instead of an authenticated mode such as GCM for implementation simplicity on the microcontroller. Authenticity is handled separately using a control-value scheme and, in exports, a fixed verification identifier encrypted as the first block. This is a recognized limitation (section 21).

PBKDF2 is applied without shortcuts: there is no inexpensive intermediate hash for checking a candidate PIN before deriving the full key. Each attempt pays the full cost of 50,000 iterations.

> *Revision 2 correction: decryption now checks that the ciphertext length is a multiple of the block size and that every PKCS#7 padding byte is consistent before accepting data. This closes a possible out-of-bounds access on corrupted records and reduces exposure to a partial padding oracle.*

# 6. Key Management

The 256-bit key derived from the PIN exists only in RAM and is never written to Flash, NVS, or SD in plaintext.

- It is created after the PIN is successfully checked against the stored control value.
- It remains in RAM while the vault is unlocked.
- It is destroyed through explicit memory cleanup on lock, auto-lock timeout, and factory reset.
- A factory reset (from brute-force protection or Lost Mode) also zeros the key before rebooting.

### Deep Sleep

The firmware does not explicitly clear the key in software before deep sleep because ESP32-S3 deep sleep powers down main RAM. Only slow and fast RTC memories survive, and the key resides in ordinary RAM. On wake, the chip boots from scratch and shows the PIN screen. Hardware power-down destroys the key.

Immediately before deep sleep, the display, touch controller, and SD card are explicitly powered off because they do not turn off automatically. This does not change key security, but significantly reduces sleep power consumption.

# 7. Authentication Flow

When a PIN is entered, the device ALWAYS derives two candidate keys—one using the decoy salt and one using the real-PIN salt—before making a decision. It then checks each against its control value. Both paths perform the same number of PBKDF2 derivations, so response time does not reveal which PIN was tried or which vault was opened.

> *Revision 2 correction: previously, the decoy was derived first and the real PIN was derived only if the decoy did not match. A valid decoy PIN therefore took half as long as a real one, revealing by timing whether the decoy vault had opened. Both paths now have the same cost.*

### What happens when the PIN is incorrect?

The persistent failure counter is incremented and saved BEFORE the attempt is evaluated, and it resets only after a confirmed successful unlock. The first three failures report how many attempts remain; the fourth shows a final-attempt warning. After five consecutive failures, the firmware starts a factory reset and reboots.

> *Revision 2 correction: previously, the counter was written after the attempt, so a power loss between verification and saving could grant extra attempts. Reserving the attempt in advance closes this bypass.*

### What happens on the fifth failure?

Immediate factory reset: erases the vault, categories, interface preferences, attempt counter, and decoy-vault remnants, but not the Factory Reset Protection record, which is intentionally retained.

# 8. Brute-force Protection

Five consecutive failures is intentionally a low threshold. An attacker with the physical device but no PIN has very little time to guess before its contents are lost; five attempts are sufficient for normal use by the legitimate owner.

A temporary lockout alone cannot stop an attacker who has unlimited time with the device. Erasing the contents makes the brute-force attack pointless: there is nothing left to decrypt. After the erase, the device cannot be immediately reconfigured because FRP requires the original PIN.

> *The effectiveness of this protection depends on hardware configuration and Flash protection. Provisioning on one unit must not be treated as a guarantee for every unit.*

## 9. Factory Reset Protection (FRP)

Factory Reset Protection prevents a factory reset—intentional or forced by an attacker—from turning a stolen device into a clean device ready for reconfiguration.

- When the PIN is first set, it is also stored with its own salt and derivation in a separate namespace that is never erased by factory reset.
- After any reset, the device detects the FRP record and the absence of an active PIN, then requires the original PIN before allowing a new one to be configured.
- Three failed attempts at the FRP PIN permanently lock the device. The counter is persistent and survives power loss and reboot.
- The only way to recover a permanently locked device is to reflash the complete firmware, erasing all Flash and making recovery of previous contents impossible.

> *Revision 2 correction: previously, the FRP attempt counter lived only in volatile memory, so cutting power after each attempt allowed unlimited retries. It is now persisted and incremented before verification.*

## 10. Decoy PIN

Designed for physical coercion. If someone forces the user to unlock the device, the decoy PIN opens a different, visually identical but empty vault, without the interface revealing that a real vault exists behind another PIN.

The decoy PIN is derived and verified like the real PIN, using its own independent salt and constant-time comparison with the real path. In decoy mode, save, edit, and delete operations report success to the interface but write nothing to persistent storage. The decoy vault always looks functional and never accumulates real data.

Decoy mode includes a configurable auto-reset timer (five minutes by default) that clears simulated remnants of the decoy session after a later real unlock.

## 11. Lost Mode (Dead Man’s Switch)

Intended to prevent a lost or forgotten device from remaining available indefinitely for someone to make unlimited attempts.

Only a successful real-PIN unlock counts as activity; a decoy unlock does not. A timestamp is recorded each time. A check runs once at each boot, before the PIN screen, using the dedicated RTC as its sole time source. If the configured number of days has passed since the last real unlock, a factory reset is performed.

If the RTC loses time or reports a date earlier than the last recorded unlock, Lost Mode does not trigger. A forward jump in RTC time may trigger erasure earlier.

## 12. Snatch Lock

Covers another physical-coercion scenario: someone snatches the device while it is unlocked. The decoy PIN does not cover this case because the real vault is already open.

The firmware periodically reads the six-axis inertial sensor and calculates combined acceleration and angular-velocity magnitudes. A real snatch produces a strong peak in both, unlike walking, putting the device away, or carrying it in a bag. If both exceed the selected sensitivity thresholds at once, the device enters deep sleep. Detection is software-based; no fixed response time is guaranteed.

Requiring both acceleration and rotation reduces false positives: an accidental impact may cause acceleration without the corresponding rotation. Detection is implemented in software because the chip's low-power mode covers only the accelerometer and cannot combine it with the gyroscope. Snatch Lock does not act during a sensitive operation (save, export, import, or vault creation). It is disabled by default and offers three sensitivity levels.

## 13. Export and Import

The encrypted export format is a custom binary container with a header containing a format identifier, 16-byte salt, iteration count, and 16-byte IV, followed by AES-256-CBC ciphertext. The first encrypted block is a fixed verification identifier: if it decrypts to the expected value during import, the export password is correct; otherwise the file is rejected.

Each exported entry includes its name, type, username, password, secret, folder, notes, and Base32 TOTP secret. A restore therefore also recovers the second factor: generated codes match the service because TOTP depends only on the seed and clock. Unencrypted CSV therefore contains passwords and 2FA seeds in plaintext; the interface warns about this, and the file must be handled as carefully as the vault itself.
Both operations show actual progress and warn against powering off or restarting during processing. The firmware also prevents deep sleep—by button or auto-lock—while an import or export is in progress, avoiding truncated files on the card.
Both operations stream data line by line directly to or from the SD card. A complete plaintext file is never materialized in persistent storage; only small fragments exist briefly in RAM. The export password is independent of the device and decoy PINs and is set by the user at export time.

### Import robustness

- The importer rejects files whose headers specify fewer than 100,000 iterations, preventing a manipulated file from forcing a cheap derivation that could facilitate an offline attack on the export password.
- The CSV parser handles lines with more fields than expected without writing outside the field array, even if a line contains many separators.
- Lines longer than the buffer are discarded in a controlled way instead of being split into garbage rows.
- Entries whose sensitive fields exceed storage limits are counted as errors instead of being silently truncated and saved as partial credentials.
- Import stops cleanly when the vault is full instead of repeatedly trying and failing on every remaining line.
- Category creation during import respects system limits, preventing a file with thousands of unique categories from exhausting storage.

### Limits of Flash Erasure

Secure erasure on Flash media (including SD cards) has known limits: wear-leveling and block remapping may leave physical copies of deleted data until overwritten. Huskey Pocket minimizes exposure by avoiding plaintext writes through encrypted streaming, but cannot guarantee deterministic physical erasure of external media.

## 14. Firmware Updates

Secure Boot is enabled on the author's unit. Application images must be signed with a key authorized by the device.

- The application image must be signed with the Secure Boot-authorized key before installation.
- There is no automatic rollback if firmware is defective. If a signed image fails, recovery requires manually reflashing an earlier version using the same signed process.

Updates to a unit with Secure Boot require an image signed with that unit's authorized key. The reference Arduino build is unsigned. This repository does not include HuskeyFlasher; do not load a binary without verifying its signature, Flash configuration, and download-mode access method. USB mass-storage mode exposes the microSD card and does not update firmware.

## 15. Physical Protection

| Scenario | Current status |
|---|---|
| Flash extraction | Secure Boot is enabled on the author's unit. Recovery availability depends on each unit's eFuses and configuration; Flash Encryption has not been verified. |
| RAM reading | The derived key exists in RAM only during an unlocked session. A live RAM-reading attack could expose it, but this is a lab-level vector outside the everyday threat model. |
| Debugger connection | Secure Boot verifies authorized images. Download and debugging availability depend on the device's eFuses and configuration. |
| Desoldering components | No specific protection against laboratory chip desoldering and analysis. This is outside the current mitigations, as is common for devices in this class. |

## 16. Bluetooth

Huskey Pocket exposes only the BLE keyboard HID profile. It exposes no other profile or vault-specific service. It sends individual keystrokes only when the user explicitly requests typing a username, password, secret, or TOTP code to the paired computer.

### Pairing and trust

- Pairing uses Numeric Comparison: both devices show the same six-digit code, which the user must explicitly confirm on the Huskey Pocket screen.
- Keystrokes are transmitted only over an encrypted and authenticated link. Until secure pairing is complete, the device types nothing, even if a physical connection exists.
- If a remote device tries to force a fixed-passkey pairing method, Huskey Pocket does not expose a predictable code; insecure pairing fails instead of completing.

- BLE advertising is active only while the vault is unlocked. When locked, the device does not advertise or accept pairing.
- A feature can forget all bonds: it disconnects paired devices cleanly, clears persistent bond storage while the stack is off, and restarts with an empty table. As with any BLE peripheral, the other device retains its side of the bond and must also be removed from its Bluetooth list.
    - Bluetooth stack events run in their own task. The pairing interface is never manipulated from that task; events queue requests for the main loop, avoiding race conditions between the stack and graphics engine by design.

The Bluetooth link never transmits the encrypted vault, PIN control values, derived keys, salts, or internal storage structures. Only the keystrokes a regular physical keyboard would send are transmitted.

> *Code changes from earlier versions: the firmware requires an authenticated link for BLE typing, validates USB blocks, and persists attempt counters. This description is not a security audit.*

## 17. USB Storage Mode

USB mass-storage mode is exclusive: while active, the vault, Bluetooth, and HID keyboard are not initialized. The device acts only as a storage drive for the SD card. It is enabled in Settings while the vault is unlocked using a persistent flag checked at boot.

### Vault isolation

The vault partition (encrypted records and control values) resides on the ESP32's internal Flash, not the SD card. USB mode exposes only the SD card to the host. A connected computer therefore cannot see the vault partition; it can access only import/export files the user placed on the card.

### Host robustness

- Block read/write requests are checked against the card's actual capacity; the host cannot read or write beyond the media limits.
- Malformed transfers (unaligned size or unsupported offset) are rejected instead of risking data corruption.
- Optional read-only mode lets the host copy SD backups but prevents overwriting or deleting them.

Residual exports are cleaned on the next normal boot, after the SD card is released from USB, so the firmware does not write to a card while it is mounted by the host.

## 18. TOTP and Diagnostic Policy

Huskey Pocket can store TOTP secrets and generate six-digit second-factor codes (time-based HMAC-SHA1, per the TOTP standard), typing them to a computer like a password. The TOTP secret is encrypted like any other sensitive field.

> *Revision 2 correction: an earlier firmware version logged the plaintext TOTP secret, its derived key, and current code over the serial port. A TOTP secret is the persistent seed for the second factor; exposing it compromises 2FA for the long term. No secret, key, or code is now logged through any diagnostic channel.*

As a general policy, firmware does not send sensitive material over the serial port: no PINs, passwords, secrets, derived keys, or generated codes. Diagnostic logs are limited to status events without secret content.

### Password generation

The password generator uses the ESP32-S3 hardware random-number generator. Character selection uses unbiased sampling (rejecting values that would introduce modulo bias), so every character in the selected set is equally likely. Requested length is bounded by the output buffer.

## 19. Scope and Design Decisions

Huskey Pocket deliberately excludes features offered by some other devices in its category because each would expand the attack surface or cost without proportional benefit for its intended use:

- No clipboard. The device does not copy credentials to a clipboard; it types them directly into the target field as a HID keyboard. Any process can observe the host clipboard, while direct typing does not leave a resident copy for malware to read later.
- No NFC/RFID. A proximity radio channel would add hardware, firmware, and a contactless access vector, conflicting with the principle that every operation requires deliberate physical interaction with the screen.
- No FIDO2/WebAuthn. Building a certifiable authenticator is beyond the scope of a handmade device in a run of about ten units; a partial FIDO2 implementation could falsely imply equivalence to dedicated, audited keys.
- microSD. In firmware 1.0.1, the card mounts in the background after the access screen appears. Files in /import are preserved during startup.
- Minimal vibration. Haptics are limited to occasional interaction confirmations and are not used as an information channel.

## 20. Stability and Memory Verification

Two AI-assisted memory reviews were conducted during development to look for leaks and memory-management defects. The author reports 16 successful tests on the code evaluated at the time. The full test cases and session records are not included in this repository; the results are therefore presented as background information, not as validation of version 1.0.1.
Firmware 1.0.1 fixes pass regression tests in a desktop test environment.

## 21. Limitations

This section exists because a security system that does not acknowledge its own limits creates more distrust than one that states them clearly.

- No system is invulnerable. This document describes reasonable mitigations for realistic threats, not absolute guarantees.
- Secure erasure on Flash media (internal and SD) has known physical limits due to wear-leveling across cells.
- The PIN has six digits and uses PBKDF2-HMAC-SHA256 with 50,000 iterations. Resistance to physical access also depends on the protections configured on each chip. Secure Boot verifies firmware signatures; it does not encrypt Flash. Flash Encryption status has not been verified on this device.
- The device does not protect against social engineering directed at the user outside the hardware.
- Encryption uses AES-CBC without a per-block authentication code. Integrity relies on the control-value scheme and, for exports, the fixed verification identifier—not on an authenticated mode such as AES-GCM.
- Secure Boot is enabled on the author's unit. Flash Encryption status has not been verified, and this document does not describe a transition to Release mode.
- The firmware has no OTA updater; images for Secure Boot units must be signed and installed through a compatible method.

- Snatch Lock is implemented in software by periodically reading the sensor, not by a dedicated hardware engine.
- A forward jump in RTC time may trigger Lost Mode earlier. The system favors failure toward erasure (data loss rather than exposure).
- Huskey Pocket has not received a security audit. AI-assisted reviews and functional tests are not a substitute for a security assessment.

## 22. Revision 4 Changes

This section summarizes the documented security state of the author's device. Secure Boot is enabled; Flash Encryption status has not been confirmed.

- Secure Boot V2 enabled: the bootloader and application are signed with a project-owned private key (RSA-PSS); the device rejects unsigned images or images signed with another key before execution.
- Flash Encryption: its status has not been checked on this device.
- Flash Encryption status depends on each unit's configuration. Earlier references to Development and Release modes do not confirm this device's state.
- Secure Boot requires applications to be signed with a key authorized by the unit. The reference Arduino binary in this repository is unsigned. This repository does not include the HuskeyFlasher updater or implement OTA. USB MSC mode provides access only to the microSD card.
- Provisioning new units: the signed bootloader and partition table are written in plaintext during the first flash of each unit, allowing the chip to complete its auto-encryption cycle; this step is not repeated during later updates.
- Data-reset feature: the flashing tool includes an option to clear data partitions (vault and settings) without touching the bootloader or firmware, useful for refurbishing a unit without repeating full provisioning.
- Boot compatibility correction: a conflict was found between the partition-table address required by a bootloader with Secure Boot V2 enabled and the fixed address used by precompiled Arduino IDE libraries. The bootloader was adjusted to fit within the original 32 KB limit, avoiding a change to the partition-table location.

## 23. Revision 3 Changes

- Complete second-factor backups: encrypted and plaintext exports now include each entry's notes and TOTP secret, and import restores them.
- Notes-only entries: records without a secret, using the note as the main content, are supported in forms, details, and backups.
- Bluetooth bond management restored (effective deletion through the stack's storage) and pairing without task race conditions.
- Deep sleep and auto-lock respect imports and exports in progress.
- The PIN is cleared from memory immediately after use during initial setup as well.
- AI-assisted memory reviews and functional tests (section 20); these are not a security audit.
- Documented behavior: microSD mounts in the background after the access screen appears and preserves /import files. The PIN uses PBKDF2-HMAC-SHA256 with 50,000 iterations; .hky backups use 100,000.

## 24. Earlier Code Changes

Summary of firmware changes included since earlier revisions, grouped by area.

### Authentication and PIN

- Constant response time between real and decoy PINs (removes a timing leak).
- Failed-attempt counter persistently reserved before each verification (removes the power-loss bypass).
- Persistent FRP attempt counter (removes unlimited retries by rebooting).

### Cryptography and storage

- Validation of block length and complete PKCS#7 padding during decryption.
- Cleanup of decoy-vault remnants during factory reset.

### Data import

- CSV parser handles lines with excess fields; oversized lines are discarded in a controlled way.
- Reject credentials that would be truncated; stop cleanly when the vault fills; enforce the category limit.
- Reject export files with an iteration count below the minimum.

### Bluetooth

- Transmit only over an encrypted and authenticated link; no fixed pairing code; safe memory handling on disconnect; effective bond deletion.

### USB storage

- Validate block ranges; reject malformed transfers; optional read-only mode.

### Diagnostics and generation

- Remove all secret leakage over the serial port, including TOTP secrets and codes.
- Password generator avoids modulo bias and bounds output length.

## 25. Frequently Asked Questions

### Why is there no cloud?

Because cloud services add a required third party and a permanent remote attack surface. Huskey Pocket does not depend on an external service. If the device is off or lost, there is no server where someone else can try to access the vault.

### Why only six digits?

The PIN has six digits. The failure counter limits consecutive attempts before a reset.

### What if I forget my PIN?

There is no backdoor or recovery method that bypasses the PIN: any recovery mechanism would be a second way into the vault and therefore an additional attack vector. Without the PIN, the only option is a factory reset—which erases everything—followed by FRP verification with the original PIN. Three incorrect FRP PIN attempts cause a permanent lock. This repository does not describe a recovery process for that state.

### If someone breaks their own device, does it affect others?

It does not compromise other devices: each derives its key from its own PIN and salts, with no global master key. A compromised device does not automatically grant access to another. Secure Boot is enabled on the author’s unit; Flash Encryption status depends on each unit.

# 26. Glossary

| Term | Definition |
|---|---|
| Salt | Unique random value combined with the PIN before deriving a key so that two vaults with the same PIN do not produce the same key. |
| IV | Initialization vector: a random 16-byte block that initializes CBC encryption. It must be unique for each operation and never reused with the same key. |
| PBKDF2 | Password-based key-derivation function that applies many hash iterations to deliberately increase the cost of each PIN-guessing attempt. |
| AES-256-CBC | 256-bit block cipher in Cipher Block Chaining mode. |
| PKCS#7 | Padding scheme that fills the final block; full validation during decryption detects corrupted or manipulated data. |
| FRP | Factory Reset Protection: requires the original PIN after a factory reset before allowing device setup. |
| TOTP | Time-based one-time password; six-digit second factor derived from a secret and the current time. |
| Deep Sleep | ESP32-S3 low-power mode that powers down most peripherals and main RAM; the device starts from scratch on wake. |
| RTC | PCF85063 clock; requires an external backup battery separate from the main battery to retain time while off. |
| IMU | Inertial measurement unit combining an accelerometer and gyroscope to measure motion and orientation. |
| Numeric Comparison | BLE pairing method in which both devices display the same code for the user to confirm. |
| Secure Boot V2 | ESP32-S3 mechanism that verifies the bootloader and application digital signatures at each boot, rejecting unsigned images or images signed with a different key. |
| Flash Encryption | Hardware encryption of the entire Flash contents using a key generated internally by the chip and never exposed to software. |
| eFuse | One-time electronic fuse inside the chip; once burned, the change is permanent and irreversible. Secure Boot and Flash Encryption are enabled and controlled through eFuses. |
| Development / Release mode | Two Flash Encryption and Secure Boot configurations. Development retains limited plaintext reflashing for recovery during development; Release permanently removes this option after the first successful boot, leaving no margin for error. |